

TALLER DE PROGRAMACION



Cómo vamos a Trabajar

Planteo de un tema

→ Conceptos teóricos

Ejercicios prácticos

Conclusiones

Formulario de preguntas

Conceptos de la clase de hoy

1

Recursión

2

Ejemplos de
Algoritmos
Recursivos

3

Conclusiones

Conceptos iniciales- RECURSION

➡ Qué podemos notar?

```
Procedure imprimir (L:lista);  
Begin  
  while (L <> nil) do  
    begin  
      write (L^.dato);  
      L:= L^.sig;  
    end;  
End;
```

Type

```
lista= ^nodo;
```

```
nodo = record  
  dato:integer;  
  sig: lista;  
end;
```

Imprimir el
contenido de la
lista



1 Imprimo el primer elemento

2 Avanzo en la lista

Lista vacía

Conceptos iniciales- RECURSION

➡ Qué podemos notar?

Calcular la potencia de un número

$$\text{num}^n = \text{num} * \text{num} * \text{num} \text{ (n veces)}$$

```
Procedure potencia (num,n:integer;var pot:integer);
```

```
Begin
```

```
  if (n = 0) then pot:= 1
```

```
  else
```

```
    begin
```

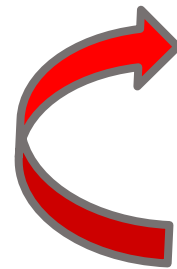
```
      pot:= 1;
```

```
      for i:- 1 to n do
```

```
        pot:= pot * num;
```

```
      end;
```

```
End;
```



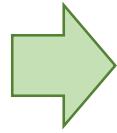
1 Multiplico el número por sí mismo

2 Se incrementa i

El número sea $i = n$

Conceptos iniciales- RECURSION

RECURSION



La recursión es una metodología para resolver problemas.

Permite resolver un problema P por resolución de instancias más pequeñas $P_1, P_2, \dots P_n$ del mismo problema.

El problema P_i es de la misma naturaleza que el problema original, pero en algún sentido es más simple.

CASO BASE: siempre existe. No debe resolverse recursivamente. En ocasiones no hay que hacer nada, sólo sirve para cortar la recursión.

Conceptos iniciales- RECURSION

ALGORITMO ITERATIVO

```
Procedure imprimir (L:lista);  
Begin  
  while (L <> nil) do  
    begin  
      write (L^.dato);  
      L:= L^.sig;  
    end;  
End;
```

ALGORITMO RECURSIVO

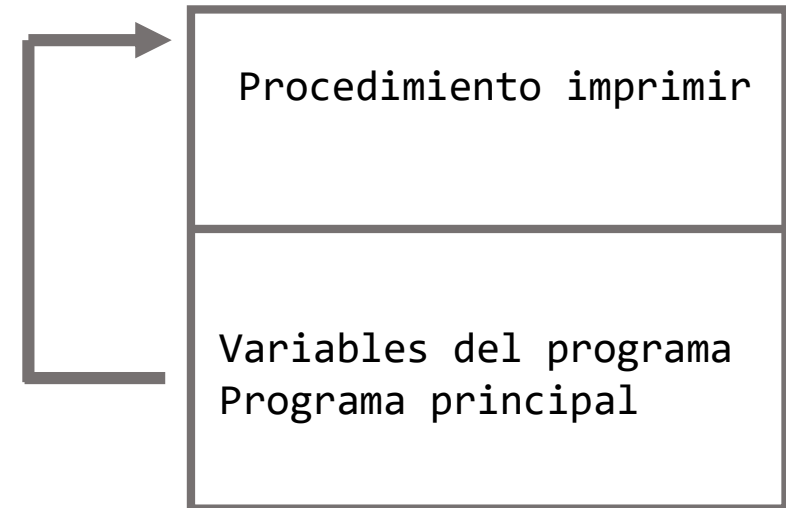
```
Procedure imprimir (L:lista);  
Begin  
  if (caso Base) then  
    código para resolver caso base {opcional}  
  
  else  
    resuelvo “algo del problema”  
    “achico” el problema  
    invoco a imprimir(parámetros)  
End;
```

Recursión- FUNCIONAMIENTO

ALGORITMO ITERATIVO

```
Procedure imprimir (L:lista);  
Begin  
  while (L <> nil) do  
    begin  
      write (L^.dato);  
      L:= L^.sig;  
    end;  
End;
```

MEMORIA



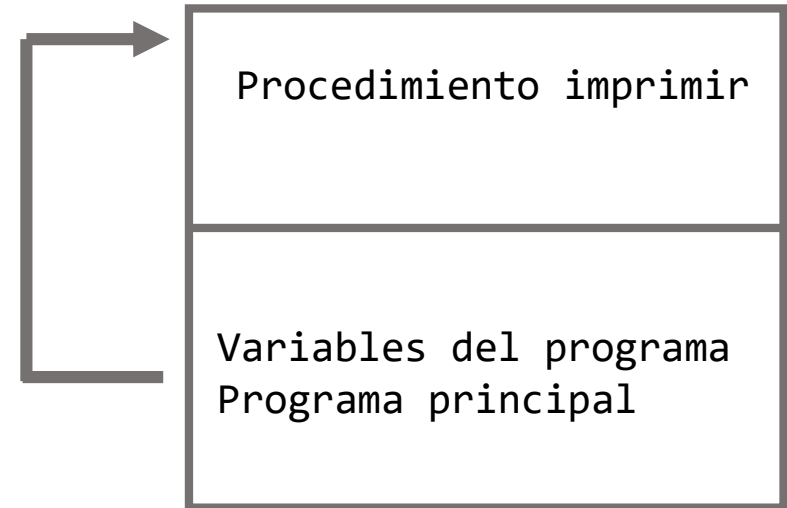
Cuando se invoca el procedimiento se carga en memoria su Código, sus parámetros y variables locales

Recursión- FUNCIONAMIENTO

ALGORITMO RECURSIVO

```
Procedure imprimir (L:lista);  
Begin  
  if (L <> nil) do  
    begin  
      write (L^.dato);  
      L:= L^.sig;  
      imprimir (L);  
    end;  
End;
```

MEMORIA



Cuando se invoca el procedimiento se carga en memoria su Código, sus parámetros y variables locales

Recursión- FUNCIONAMIENTO - Ejemplo

Ejercicio: Realice un programa que lea un número num invoque a un procedimiento imprimir que dado el número leído imprime todos los números desde num hasta 1.

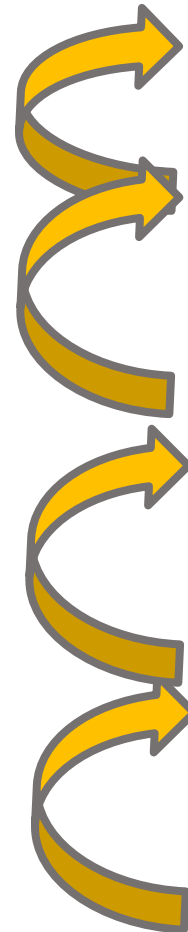
```
Program uno;  
Var  
  num:integer;  
  
Begin  
  read (num);  
  if (num > 0) then  
    imprimir(num);  
End.
```

Recursión- FUNCIONAMIENTO - Ejemplo

Ejercicio: Realice un programa que lea un número num invoque a un procedimiento imprimir recursive que dado el número leído imprime todos los números desde num hasta 1. **Supongamos que se lee num = 4**

```
Procedure imprimir (num:integer);
Begin
  IF (num > 0) then
    begin
      write (num);
      num:= num - 1;
      imprimir (num);
    end;
End;
```

Parámetros?



Imprimir num = 4, **imprime 4**
num := 3

Imprimir num = 3, **imprime 3**
num := 2

Imprimir num = 2, **imprime 2**
num := 1

Imprimir num = 1, **imprime 1**
num := 0

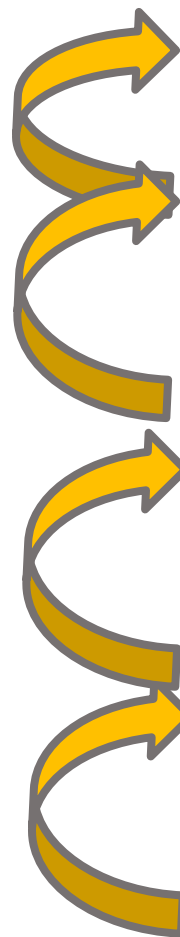
Imprimir num = 0, **no hace nada**

Recursión- FUNCIONAMIENTO - Ejemplo

Ejercicio: Realice un programa que lea un número num invoque a un procedimiento imprimir recursivo que dado el número leído imprime todos los números desde num hasta 1. Supongamos que se lee num = 4

```
Procedure imprimir (VAR num:integer);
Begin
  IF (num > 0) then
    begin
      write (num);
      num:= num - 1;
      imprimir (num);
    end;
End;
```

Num = 0 cuando vuelve al programa principal



Imprimir num = 4, **imprime 4**
num := 3

Imprimir num = 3, **imprime 3**
num := 2

Imprimir num = 2, **imprime 2**
num := 1

Imprimir num = 1, **imprime 1**
num := 0

Imprimir num = 0, **no hace nada**

Recursión- FUNCIONAMIENTO - Ejemplo

Ejercicio: Realice un programa que dada una lista e invoque a un procedimiento recursive que imprima los elementos de la lista

Supongamos que la lista se generó con los valores 4 10 7

Programa dos:

Type

```
    lista= ^nodo;  
    nodo = record  
        dato:integer;  
        sig: lista;
```

end;

Var

```
    pri:lista;
```

Begin

```
    cargarLista (pri);  
    imprimir (pri);
```

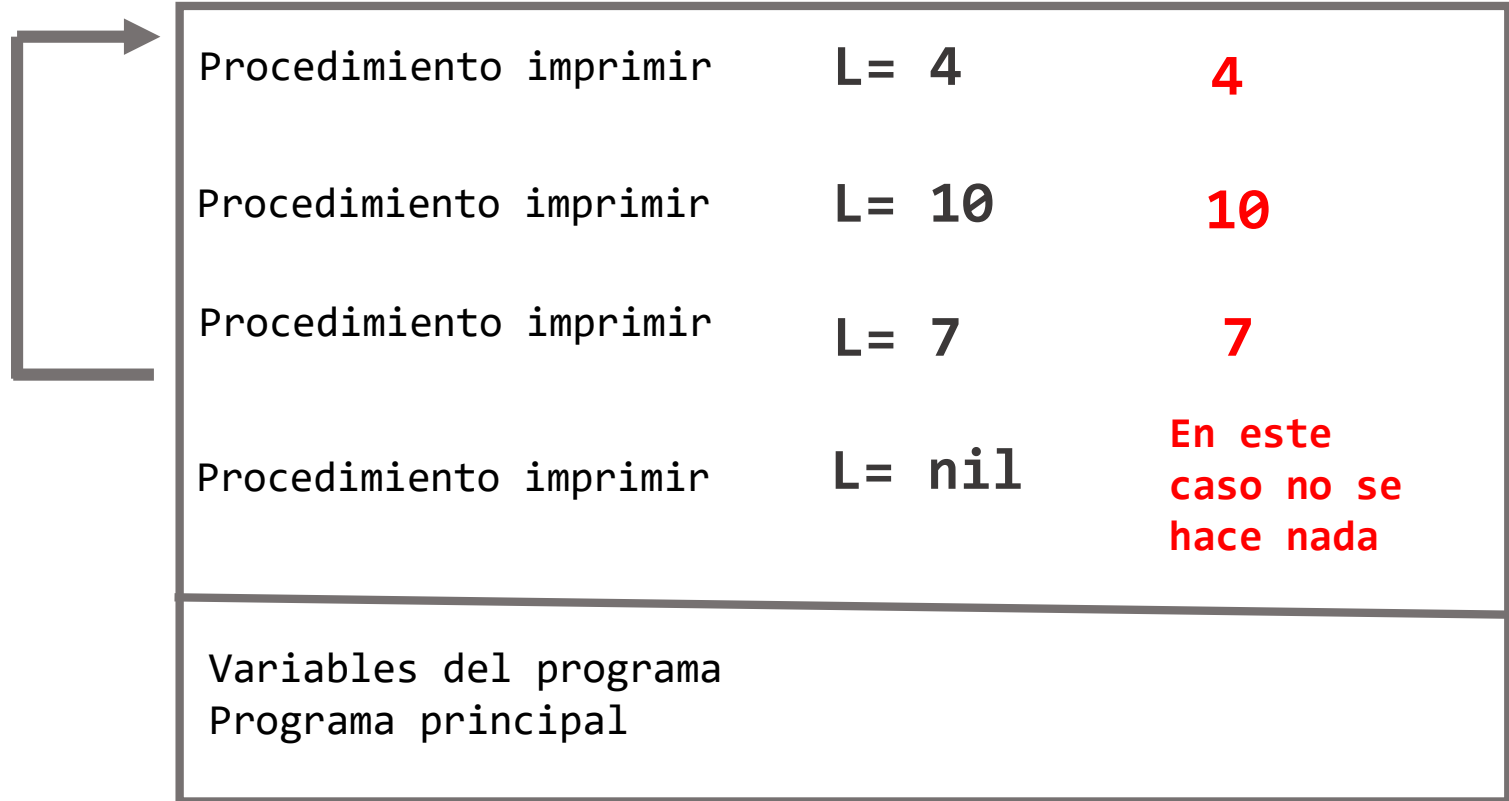
End.

Recursión- FUNCIONAMIENTO - Ejemplo

Ejercicio: Realice un programa que dada una lista e invoque a un procedimiento recursive que imprima los elementos de la lista

Supongamos que la lista se generó con los valores 4 10 7

```
Procedure imprimir (L:lista);  
Begin  
  IF (L <> nil) then  
    begin  
      write (L^.dato);  
      L:= L^.sig;  
      imprimir (L);  
    end;  
End;
```



Procedimiento imprimir	L= 4	4
Procedimiento imprimir	L= 10	10
Procedimiento imprimir	L= 7	7
Procedimiento imprimir	L= nil	En este caso no se hace nada

Variables del programa
Programa principal

Recursión- FUNCIONAMIENTO - Ejemplo

Suponga que debe realizar un módulo recursivo que dado un número entero y un valor n (entero positivo) se calcule x^n .

Supongamos $x= 3$ y $n = 2$.

Programa tres:

Var

$x, n, \text{resultado} : \text{integer};$

Begin

 read (x);

 read (n);

 potencia ($x, n, \text{resultado}$);

 write (resultado);

End.

Programa tres:

Var

$x, n, \text{resultado} : \text{integer};$

Begin

 read (x);

 read (n);

$\text{resultado} := \text{potencia} (x, n);$

 write (resultado);

End.

Recursión- FUNCIONAMIENTO - Ejemplo

Suponga que debe realizar un módulo que dado un número entero y un valor n (entero positivo) se calcule x^n .

Supongamos $x= 3$ y $n= 2$.

```
Function potencia (num,n:integer): integer;
```

```
Var
```

```
    pot:integer;
```

```
Begin
```

```
    if (n = 0) then potencia:= 1
```

```
    else
```

```
        potencia:= num * potencia(num, n-1));
```

```
    end;
```

```
End;
```

Supongamos num = 3 n=2

9

potencia num= 3,n=2, $3 * \text{potencia}(3,1)$

3

9

potencia num= 3,n=1, $3 * \text{potencia}(3,0)$

1

3

potencia num= 3,n=0, $\text{potencia} = 1$

Recursión CONCLUSIONES



Hay problema de naturaleza recursiva, NO todo se puede resolver de manera recursiva.

Requiere más memoria que una solución iterativa.

Siempre hay al menos un caso base, a veces se implementa código y otras es sólo para cortar la recursión.

Hay que "achicar " el problema en cada llamado.

No es lo mismo que los parámetros estén pasados por valor o por referencia

Formulario